

Resumen teórico — Sistemas Operativos

Notas del curso: *Entrada/Salida*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo *Entrada/Salida* de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC). Se presentan la clasificación de los dispositivos, las formas de interconectarlos a la CPU (bus de E/S dedicado y *memory-mapped I/O*), el modelo de control basado en puertos, la arquitectura de los *device drivers*, el diseño del subsistema de E/S, las disciplinas de líneas, los buffers/caché de bloques, la planificación de requerimientos (incluido el algoritmo del elevador), la transferencia directa a memoria (DMA), la integración con el sistema de archivos (archivos especiales en `/dev`, *major/minor numbers*, `file_operations`), los dispositivos virtuales (`random`, `zero`, `null`, RAID), los *kernel modules* cargables dinámicamente y los estándares más relevantes (IDE/ATA, UART, VirtIO). El objetivo es servir como material de estudio para preparar los prácticos asociados a E/S y el final de la materia.

Índice

1. Introducción y clasificación de dispositivos	2
1.1. Por dirección de transmisión de datos	2
1.2. Por modo de transferencia	2
2. Interconexión de dispositivos	2
2.1. Ejemplo: arquitectura x86 (híbrida)	3
2.2. Ejemplo: RISC-V	3
2.3. Ejemplo: modo texto VGA	3
3. Buses de interconexión	3
3.1. Buses estándar: PCI y USB	4
3.2. Auto-detección y plug-and-play	4
4. Control de dispositivos	4
5. Device drivers	5
5.1. Funciones que debe implementar un driver	5
5.2. Polling como alternativa	5
6. Diseño del subsistema de E/S	5
6.1. Interfaz de un device driver	6
7. Software independiente de dispositivos	6
7.1. Disciplina de líneas (line discipline)	6
8. Buffers y caché	6
9. Planificación de E/S	7
9.1. Discos: algoritmo del elevador	7

10. Transferencia directa a memoria (DMA)	7
10.1. Cómo funciona el DMA	7
11. Integración con el sistema de archivos	8
11.1. La estructura <code>file_operations</code> en Linux	8
11.2. Archivos especiales en <code>/dev</code>	8
11.3. Major y minor numbers	9
12. Dispositivos virtuales	9
13. Device drivers cargables dinámicamente	10
13.1. Diseño microkernel vs. monolítico	10
13.2. Linux Kernel Modules (LKM)	10
14. Estándares de dispositivos	10
15. VirtualIO interface	11
15.1. Virtual queues	11
16. Cierre	11

1. Introducción y clasificación de dispositivos

Un sistema de computación incluye generalmente *muchos* dispositivos de entrada/salida (E/S), de distintos tipos. Hay dos formas naturales de clasificarlos.

1.1. Por dirección de transmisión de datos

- **Entrada:** sólo se pueden leer datos del dispositivo (por ejemplo, un teclado, un mouse o un sensor).
- **Salida:** sólo se pueden escribir datos hacia el dispositivo (por ejemplo, una impresora o un display).
- **Entrada-Salida:** se pueden hacer ambas operaciones (un disco, una interfaz de red).

1.2. Por modo de transferencia

- **Dispositivos de caracteres (bytes).** Transfieren de a bytes. Ejemplos: teclado, mouse, dispositivos de comunicaciones tipo UART.
- **Dispositivos de bloques.** Transfieren de a *bloques* de tamaño fijo (por ejemplo, 512 bytes). El ejemplo paradigmático es el disco.

Hay dispositivos que no encajan limpiamente en esta clasificación: por ejemplo, las interfaces de red tipo Ethernet transmiten en forma serial bloques de tamaño *variable* llamados **paquetes** (*packets*), que tienen un tamaño mínimo y uno máximo.

El timer. Otro dispositivo que se maneja de forma especial es el **timer**: se programa para generar *interrupciones periódicas* y así permitirle al kernel tomar el control con regularidad (es la base de la preemption del scheduler y del envejecimiento de bits en memoria virtual).

2. Interconexión de dispositivos

Una arquitectura puede conectar sus dispositivos a la CPU de dos maneras:

1. Mediante un **bus de E/S** dedicado y separado del bus de memoria.
2. Conectándolos directamente al espacio de direcciones de memoria: **memory-mapped I/O (MMIO)**.

La interconexión al espacio de direcciones puede ser *hard-wired*, pero las arquitecturas modernas usan un **memory controller chip** programable que permite rutear los requerimientos de lectura/escritura a distintos chips (ROM, DRAM, dispositivos de I/O) según el valor del bus de direcciones.

2.1. Ejemplo: arquitectura x86 (híbrida)

La arquitectura x86 (Intel) es **híbrida**: la CPU contiene un *bus de E/S* separado del bus de memoria. Algunos dispositivos se conectan al bus de E/S (con un espacio de direcciones de 16 bits), mientras que otros se mapean en memoria.

- Para el bus de E/S, la CPU provee instrucciones especiales: **in** y **out**.
- En una PC basada en x86, el *frame buffer* del controlador de video (CGA, VGA, etc.) está **mapeado en memoria**: al escribir datos en ese espacio reservado se produce una salida en el display.

2.2. Ejemplo: RISC-V

El board RISC-V usado en el taller no dispone de un *video controller*, pero soporta la conexión de una terminal (*tty*) por su puerto serie UART. La terminal transmite los códigos de las teclas pulsadas y muestra en pantalla los caracteres recibidos. La interfaz de programación del UART permite transmitir y recibir bytes mediante un protocolo serial muy simple.

Idea intuitiva

Las arquitecturas modernas como RISC-V tienden a usar exclusivamente **MMIO**, aprovechando los enormes espacios de direcciones lógicas disponibles. Esto simplifica el diseño del hardware y elimina la necesidad de un bus adicional.

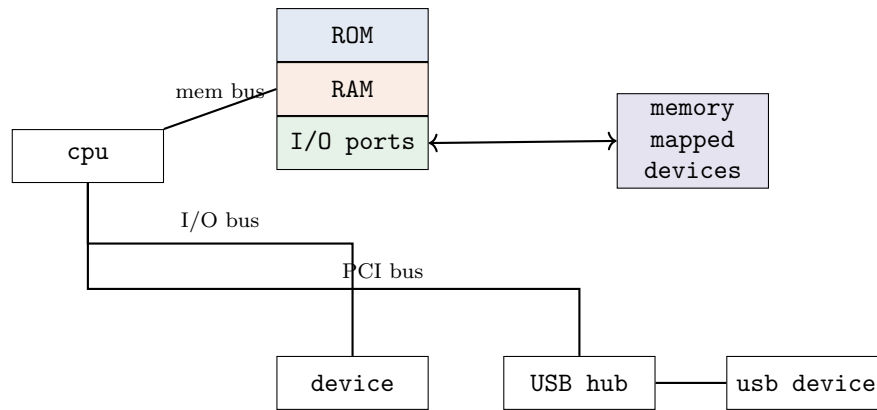
2.3. Ejemplo: modo texto VGA

El controlador VGA en la IBM PC expone el *text buffer* de video en la dirección física **0xB800**, de tamaño dependiente del modo (típicamente 80 columnas × 25 filas). Cada carácter se representa con 16 bits de la forma (*ascii, attributes*): el byte ASCII más un byte de atributos (color de fondo, color de frente y parpadeo). En DOS o en las primeras versiones de MS-Windows, las aplicaciones podían escribir directamente en este buffer, sin protección.

Modo gráfico y fonts. El hardware gráfico moderno soporta **modo gráfico** (matriz de píxeles). El modo texto usado en consolas se implementa *dibujando* los caracteres gráficamente. La forma de cada carácter se describe en un **font**, que puede ser *raw* (matriz de píxeles) o *vectorial* (paths de trazado). La representación vectorial permite un mejor escalado.

3. Buses de interconexión

La mayoría de las arquitecturas soportan la conexión de *buses de E/S*. Un bus es a su vez un dispositivo con su propio controlador programable. Permite conectar varios dispositivos y, además, otros buses secundarios.



3.1. Buses estándar: PCI y USB

Un bus estándar muy usado como bus principal es el **PCI** (*Peripheral Component Interconnect*). Permite conectar:

- interfaces de red,
- controladoras de disco,
- co-procesadores gráficos,
- dispositivos de audio,
- y a su vez otros controladores como **USB** (*Universal Serial Bus*), que soporta un árbol de *USB hubs*, donde cada hub contiene puertos en los que se conectan dispositivos físicos.

Los estándares de buses como PCI o USB describen tanto las características *físicas* (conectores, niveles eléctricos) como la *interfaz de control* (registros de control, estado y datos, y los detalles de programación).

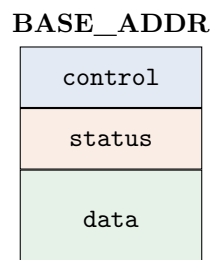
3.2. Auto-detección y plug-and-play

Cada dispositivo compatible **se anuncia en el bus** al iniciarse (o al ser conectado), informando: tipo de dispositivo, fabricante, registros de E/S, líneas de interrupción a usar, etc. El controlador del bus registra esta información en una tabla que permite al SO *auto-detectar* los dispositivos existentes.

Algunos buses permiten conexión en caliente: notifican al kernel (generando una interrupción) cuando se conecta un nuevo dispositivo, permitiendo activar el *device driver* correspondiente. Esto se conoce como **plug-and-play**.

4. Control de dispositivos

Un dispositivo expone un conjunto de **puertos** (o registros) de control, estado y datos. Estos puertos están contenidos en el **controlador del dispositivo** (*device controller*).



- **Registros de control.** Se usan para enviar comandos al dispositivo.
- **Registros de datos.** Contienen los datos a transferir (lectura o escritura).

- **Registros de estado.** Indican el estado del dispositivo: si hay un dato disponible, si se produjo un error o si el dispositivo está configurado para generar interrupciones.

Estos puertos aparecen en el espacio de direcciones del bus de E/S o en el de memoria, según la forma de interconexión. La documentación técnica del dispositivo (o del estándar si lo cumple) describe cómo debe el SO interactuar con él; las direcciones suelen ser *dependientes de la plataforma*.

5. Device drivers

Definición

Un **device driver** es el módulo del kernel que controla un tipo o familia de dispositivos. El SO define la *interfaz* (funciones) que el driver debe implementar; el kernel se comunica con el driver mediante esa API.

Diseño polimórfico. Se sigue un diseño polimórfico al estilo *orientado a objetos*: cada device driver expone una estructura de datos con *punteros* a las funciones que implementan la API. El kernel invoca esos punteros sin conocer los detalles internos del driver.

5.1. Funciones que debe implementar un driver

Comúnmente un device driver implementa:

- **Detección** (*discovery*) de los dispositivos que puede controlar.
- **Inicialización** de los dispositivos: configurar las interrupciones que generará, informar al kernel las direcciones de E/S que reservará, etc.
- **Operaciones de lectura y escritura:** implementar el control y programación de los requerimientos de transferencia (*I/O requests*).
- **Manejo de interrupciones:** cuando un dispositivo completa una operación, genera una interrupción y el driver debe atenderla.

5.2. Polling como alternativa

Una forma alternativa de detectar la finalización de una operación es **pooling** (o *polling*): verificar periódicamente un bit de un registro de estado del dispositivo.

Importante

Las interrupciones son preferibles cuando el dispositivo es “lento” (el CPU puede hacer otra cosa mientras espera). El polling es preferible cuando el dispositivo es “rápido” y se espera un dato inmediato; el costo de manejar la interrupción superaría al de simplemente leer el estado.

6. Diseño del subsistema de E/S

Dada la enorme variedad de dispositivos, el SO necesita un diseño que abstraiga a los device drivers de las capas superiores del kernel.

Cada device driver se **registra** en el sistema y reserva los recursos que requiere: espacios de direcciones para los puertos del controlador, y la línea (o número) de interrupción que usará. El subsistema de E/S mantiene entonces:

- Una **tabla de dispositivos**, cuya estructura por entrada contiene las direcciones de puertos de I/O, las líneas de IRQ y otros datos.
- Una asociación entre cada dispositivo y su *device driver*, representado por una estructura de datos con punteros a sus operaciones soportadas.

6.1. Interfaz de un device driver

Para facilitar la interoperabilidad, cada driver implementa una interfaz definida en el SO (habitualmente por el filesystem). Es común que se defina una interfaz *por cada tipo de dispositivo*. En los SO tipo UNIX hay tres clases:

1. **Dispositivos de caracteres.** La E/S es de a bytes. Ejemplo: teclado, mouse, puerto serie.
2. **Dispositivos de bloques.** La lectura/escritura es por bloques (discos, almacenamiento no volátil). El filesystem ofrece una caché para la E/S de bloques.
3. **Dispositivos de comunicaciones o red.** Ejemplo: interfaces Ethernet, Wifi, Bluetooth.

7. Software independiente de dispositivos

El subsistema de E/S provee además algunos servicios o políticas que *no dependen* del dispositivo concreto: planificación de requerimientos, manejo de buffers, etc.

7.1. Disciplina de líneas (line discipline)

Un ejemplo importante es el servicio de **disciplina de líneas** entre los *tty device drivers* y las llamadas al sistema como **read/write** sobre terminales de texto. Una disciplina típica realiza E/S de texto *por líneas* (cadenas de caracteres).

Entrada. La disciplina de entrada (*input*) tiene en cuenta caracteres de control:

- **Enter:** indica fin de la línea.
- **Ctrl-D:** indica fin de la entrada (EOF).
- **BS (*backspace*):** borra el carácter ingresado previamente.

Su implementación se basa en la gestión de un **buffer de caracteres de entrada organizado como una cola circular**.

Salida. En la salida (*output*) se interpretan caracteres de control como:

- **CR (*carriage return*):** retorno de carro.
- **LF (*linefeed*):** bajada de cursor (scroll) y avance de línea.

Idea intuitiva

En xv6, el módulo `uart.c` implementa el driver y la disciplina de líneas sobre una terminal conectada al puerto serie UART.

8. Buffers y caché

Para minimizar los accesos al dispositivo (como un disco), cada vez que se lee un bloque se carga en bloques de memoria (típicamente páginas) que se manejan como una **caché**. Esa caché tiene un límite de bloques.

Política de reemplazo. Los bloques de la caché se *reúsan*; comúnmente se usa un algoritmo **LRU** para elegir la víctima a liberar (estamos viendo la misma idea que en memoria virtual, ahora aplicada a bloques de disco).

Sincronización a disco. Las llamadas al sistema `flush(fd)` o `close(fd)` producen que se transfieran los bloques modificados (*dirty*) del archivo desde la caché al disco.

Importante

La E/S de dispositivos de *caracteres* comúnmente **no** utiliza buffers de caché: los datos se procesan a medida que llegan (por ejemplo, una pulsación de tecla).

9. Planificación de E/S

El SO debe gestionar los requerimientos de E/S generados por los procesos (y por el propio kernel). Algunos algoritmos son similares a los de planificación de procesos:

- **FIFO:** los requerimientos se procesan en orden de llegada.
- **Prioridades:** se ordenan según una prioridad asignada.

9.1. Discos: algoritmo del elevador

Para dispositivos de bloques electromecánicos (discos magnéticos), los requerimientos pueden ordenarse por la **cercanía a la posición actual del brazo** de lectura/escritura. Procesar los más cercanos reduce significativamente los tiempos de acceso. Pero esto puede hacer que requerimientos en cilindros lejanos esperen indefinidamente (*starvation*). La solución es el **algoritmo del elevador**:

Definición

El **algoritmo del elevador** mantiene una variable `direction` que representa la dirección actual del brazo. Los requerimientos se ordenan y procesan en *esa* dirección. Al llegar al cilindro del extremo, se invierte `direction` y se procesan los restantes en orden de cercanía en la nueva dirección.

Es análogo al funcionamiento de un ascensor: atiende a todos los pisos en una dirección antes de cambiar de sentido.

10. Transferencia directa a memoria (DMA)

Hay dos formas de mover datos entre la RAM y los buffers internos de un dispositivo:

1. **Por software:** el driver hace una copia byte a byte o palabra a palabra, del estilo de un `memcpy(dst, src, count)`. Esto puede insumir muchos ciclos de CPU.
2. **Por DMA** (*Direct Memory Access*): el dispositivo realiza la transferencia de manera autónoma, en paralelo con la ejecución de instrucciones de la CPU.

10.1. Cómo funciona el DMA

Para usar DMA, el device driver le indica al dispositivo:

1. La **dirección física** del buffer en memoria.
2. El **tamaño** de la transferencia.
3. La orden de iniciar.

Mientras la transferencia se realiza, el dispositivo DMA compite con la CPU por el acceso al bus de memoria mediante una técnica de **robo de ciclos** (*cycle stealing*): aprovecha los ciclos en los que la CPU no necesita la memoria.

Notificación. Cuando la transferencia finaliza, el dispositivo DMA genera una **interrupción** para avisarle al driver.

Idea intuitiva

Actualmente, muchos dispositivos modernos ya incluyen un mecanismo DMA *en el mismo chip*, por lo que el driver no necesita programar un controlador DMA externo.

11. Integración con el sistema de archivos

El sistema de E/S se integra con el sistema de archivos para brindar una abstracción central en los SO tipo UNIX:

Importante

Los dispositivos **son archivos especiales**, y los procesos acceden a ellos (a través de los drivers) mediante la API de operaciones sobre archivos: `open`, `read`, `write`, `close`, ...

11.1. La estructura `file_operations` en Linux

El filesystem define un *conjunto de operaciones* que el driver de ese tipo debe implementar. En Linux, esta interfaz toma la forma de una `struct` con punteros a función:

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    int (*readdir) (struct file *, void *, filldir_t);
    unsigned (*poll) (struct file *, struct poll_table_struct *);
    int (*ioctl) (struct inode *, struct file *, unsigned, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    ssize_t (*readv) (struct file *, const struct iovec *, ...);
    ssize_t (*writev) (struct file *, const struct iovec *, ...);
};
```

Cada driver llena esta estructura con sus implementaciones específicas.

11.2. Archivos especiales en `/dev`

En los sistemas tipo UNIX, los archivos que representan dispositivos están creados bajo `/dev`. El *tipo* de cada archivo indica la clase de dispositivo:

- `c` si es de caracteres,
- `b` si es de bloques.

Un proceso de usuario puede usar `open`, `read`, `write`, `close` sobre estos archivos para interactuar directamente con el driver del kernel. Por ejemplo:

```
$ echo hello > /dev/tty
hello
```

11.3. Major y minor numbers

En los sistemas tipo UNIX, un archivo especial contiene sólo *metadatos* (no almacena datos en disco). Los metadatos más importantes son:

- Tipo (`c` o `b`).
- **Major number**: representa el *tipo* de dispositivo y le permite al kernel identificar al device driver correspondiente.
- **Minor number**: representa el *número de dispositivo* dentro de los de su tipo. Se pasa al driver en cada operación (en un campo de la estructura `inode`), para que éste sepa con qué dispositivo concreto interactuar.

Idea intuitiva

Esta separación permite que un *único driver* maneje muchos dispositivos del mismo tipo: el major identifica al driver y el minor le dice “cuál de mis dispositivos”.

12. Dispositivos virtuales

Es común que el SO implemente dispositivos en *software* (no existen físicamente):

- **random**: generador de números aleatorios. Sólo lectura.
- **zero**: retorna ceros. Sólo lectura.
- **null**: ignora todo lo que recibe. Sólo escritura.

Otros dispositivos virtuales agrupan varios dispositivos físicos y usan sus drivers correspondientes:

Consola/terminal (tty). Una *consola* consiste en un teclado y un display. El driver de la *tty* define el control y la asociación con esos dispositivos. Un driver de este tipo suele soportar **consolas o terminales virtuales**: el SO permite al usuario crear varias terminales como espacios de trabajo distintos.

RAID. Un **RAID** (*Redundant Array of Inexpensive Disks*) es un conjunto de discos presentado al sistema como un único disco. Es un dispositivo de almacenamiento tolerante a fallas que permite accesos en paralelo:

- **Distribuye** los bloques de datos entre los discos: un archivo cuyos bloques están dispersos puede leerse en paralelo.
- **Replica** los bloques: ante la falla de un disco, existe una copia en otro y el sistema sigue funcionando.

Existe hardware específico para RAID, pero también puede implementarse por software usando discos estándar (por ejemplo, ATA). GNU/Linux soporta RAID por software.

13. Device drivers cargables dinámicamente

La mayoría de los SO modernos permiten cargar device drivers **dinámicamente**, es decir, sin necesidad de reiniciar ni recompilar el kernel. Esto significa que el driver se compila como un **kernel module**.

13.1. Diseño microkernel vs. monolítico

- En un diseño **microkernel**, esto es natural: cada driver es un componente (programa) independiente del resto del kernel y se comunica con el kernel *por mensajes*. Por ejemplo, en MS-Windows los drivers toman la forma de *Dynamic Link Libraries* (DLLs).
- En un diseño **monolítico**, es más complicado: el módulo debe enlazarse dinámicamente con el código del kernel para vivir en su *mismo espacio de memoria*.

13.2. Linux Kernel Modules (LKM)

En GNU/Linux los módulos son **shared objects** compilados y enlazados como una biblioteca compartida con extensión `.ko` (*kernel object*). Se llaman **Linux Kernel Modules (LKM)**.

- El comando `insmod` mapea el archivo objeto (driver) en memoria y ejecuta la llamada al sistema `init_module()`, que realiza el enlazado dinámico del módulo en el espacio del kernel e invoca a su función `init()`.
- GNU/Linux provee los comandos de usuario `lsmod`, `insmod` y `rmmod` para listar, cargar y remover módulos cargables dinámicamente.

14. Estándares de dispositivos

Existen varios estándares para diferentes tipos de dispositivos. Estos estándares especifican detalles *físicos* (niveles eléctricos, conectores) y *lógicos* (registros de control/estado, detalles de configuración y operación).

Idea intuitiva

Que los dispositivos adhieran a estándares simplifica drásticamente la escritura de los drivers: basta con basarse en la documentación del estándar, sin tener que adaptarse a cada fabricante.

Algunos estándares relevantes:

- **IDE** (*Integrated Drive Electronics*): interfaz estándar para dispositivos de almacenamiento masivo (discos).
- **AT Attachment (ATA)**: evolución de IDE; define dispositivos con transferencia en paralelo (**PATA**) y en serie (**SATA**).
- **UART** (*Universal Asynchronous Receiver-Transmitter*): dispositivos de comunicación serial.
- **ISO/IEC JTC**: estándares para equipos de oficina como impresoras y escáneres.
- **VirtIO**: interfaz de dispositivos *virtuales*. Pensada para SO corriendo sobre máquinas virtuales. Define diferentes tipos de dispositivos y sus interfaces *lógicas* (puertos y operación). No incluye detalles físicos, ya que su objetivo es definir dispositivos virtuales emulados por software.

15. VirtualIO interface

VirtIO define una **interfaz unificada** para diferentes tipos de dispositivos. Los dispositivos VirtIO aparecen mapeados en memoria o conectados a un bus PCI y exponen:

- su *tipo* (block, network, input device, ...),
- su *estado*,
- las *características* que soportan,
- el mecanismo de *notificaciones*,
- un espacio de *configuración*,
- y una o más **virtual queues**.

15.1. Virtual queues

Definición

Una **virtual queue** es una estructura que permite gestionar buffers de entrada/salida *compartidos* entre el device driver y el device controller (el dispositivo). Es el canal de transferencia de datos y requerimientos entre ambos.

Uso típico.

- Con dispositivos como **discos**, el driver usa *una* virtual queue con los requerimientos y buffers de lectura/escritura.
- Con dispositivos de **red**, se usan *dos* virtual queues: una para los paquetes recibidos y otra para los paquetes a enviar.

16. Cierre

El subsistema de E/S es la capa que pone en contacto al kernel con la diversidad infinita de hardware que existe afuera. La forma en que las Notas organizan esta diversidad puede resumirse en pocas ideas:

- **Clasificar** los dispositivos por dirección de transmisión y por modo de transferencia (caracteres, bloques, paquetes) permite definir interfaces uniformes.
- **Conectar** los dispositivos a la CPU vía buses dedicados o vía *memory-mapped I/O*; las arquitecturas modernas (RISC-V) se inclinan a MMIO porque aprovecha el espacio de direcciones lógicas y simplifica el hardware.
- **Programar** cada dispositivo mediante un conjunto de *puertos* de control, datos y estado, encapsulados en su controlador.
- **Abstraer** cada dispositivo con un **device driver** que implementa una API polimórfica común (detección, init, read/write, manejo de IRQ). El subsistema de E/S mantiene la tabla de dispositivos y la asociación driver ↔ dispositivo.
- **Compartir** código entre drivers mediante *software independiente del dispositivo*: disciplinas de líneas, buffer de caché de bloques (con LRU), planificación de requerimientos (FIFO, prioridades, algoritmo del elevador en discos).
- **Acelerar** las transferencias mediante **DMA**, que libera a la CPU del trabajo de copiar datos por software.
- **Integrar** todo en el filesystem: los dispositivos son archivos especiales en `/dev`, con *major/minor numbers*, accesibles vía la API estándar `open/ read/write/close`; en Linux la `struct file_operations` concreta esta idea.

- **Extender** el sistema con dispositivos virtuales (`random`, `zero`, `null`, tty virtuales, RAID por software) y con módulos cargables dinámicamente (LKM, `insmod`, `rmmmod`).
- **Estandarizar** las interfaces de hardware (IDE/ATA, UART, VirtIO) para simplificar la escritura de drivers; VirtIO en particular hace lo mismo pero para dispositivos virtuales en máquinas virtuales, usando *virtual queues*.

Idea intuitiva

La gran idea es la **uniformidad**: todo lo que pasa en E/S, por distinto que sea el dispositivo, termina presentándose al resto del kernel y al usuario como “otro archivo”. Ese principio Unix — “todo es un archivo” — es lo que hace manejable la complejidad inmensa del hardware moderno.